

CSE232 시스템프로그래밍(캡스톤디자인)

5주차 첫 번째 수업

# 시스템 정보와 자원 한계

경민기

# 수업 구성

---

복습과 1부 시스템 식별 · 30분

2부 sysconf와 경계값 · 25분

3부 자원 한계 · 30분

4부 /proc 관찰 창 · 20분

확인 퀴즈와 제출 · 5분

120분 수업 · 1부 후 휴식 10분

# 왜 시스템 정보를 읽는가

---

실행 환경은 매번 다르다: 커널, CPU, 메모리, 열린 파일 상한.

진단 로그: 오류 보고에 시스템 식별 정보를 함께 남긴다.

이식성: 컴파일할 때 정하지 않고 실행할 때 값을 얻는다.

자원 계획: 버퍼 크기나 동시 작업 수를 한계 안에서 정한다.

2주차에 `uname·sysconf` 예제를 잠깐 실행했습니다. 오늘은 계약까지 다룹니다.

# 1부

## 시스템 식별

uname과 sysinfo로 실행 환경을 확인합니다.

# uname(2)과 struct utsname

---

## **sysname / nodename**

커널 이름과 네트워크상의  
호스트 이름이다.

## **release / version**

커널 릴리스와 빌드 정보  
문자열이다.

## **machine**

CPU 아키텍처 이름이다.  
x86\_64, aarch64 등이다.

헤더: <sys/utsname.h>. 성공 0, 실패 -1입니다.

# sysprobe: uname 조회

```
#include <sys/utsname.h>

struct utsname u;
if (uname(&u) == -1) {
    perror("uname");
    return 1;
}
printf("sysname=%s\n", u.sysname);
printf("nodename=%s\n", u.nodename);
printf("release=%s\n", u.release);
printf("machine=%s\n", u.machine);
```

## 호출 결과

성공 0, 실패 -1이다.  
실패하면 errno를 본다.

## 문자열 필드

구조체 안에 들어 있는  
널 종료 문자열이다.

## 출력 시점

조회에 성공한 뒤에만  
필드를 읽는다.

# uname(1) 명령과 비교

```
$ uname -a
Linux lab-ubuntu 6.8.0-49-generic #49-Ubuntu \
SMP ... x86_64 GNU/Linux

# -s Linux          -n lab-ubuntu
# -r 6.8.0-49-generic -m x86_64

$ uname -r
6.8.0-49-generic
$ uname -m
x86_64
```

출력 예시: Ubuntu 22.04 환경입니다.

## 명령과 함수

uname(1)은 같은 정보를 셸에서 확인한다.

## 함수의 쓰임

프로그램 안에서 로그나 환경 판별에 사용한다.

## 출력 차이

버전 문자열 형식은 배포판마다 다르다.

# sysinfo(2): 자원 요약

---

## 구조체 필드

uptime, loads, totalram, freeram, procs 등이 있다.

## 단위 환산

램 필드 값에 mem\_unit을 곱해 바이트로 만든다.

## Linux 전용

표준 인터페이스가 아니므로 이식성 코드와 구분한다.

헤더: <sys/sysinfo.h>. Linux 전용입니다.

# 실습 1: 시스템 식별 관찰

```
./sysprobe
uname -a
uname -r
uname -m

# Ubuntu에서 시스템 호출 관찰
strace -e uname ./sysprobe

# Linux에서만 동작
./sysinfo_demo
```

## uname 비교

sysprobe 출력과 uname 명령의 열을 맞춘다.

## 호출 관찰

strace로 uname 시스템 호출을 확인한다.

## 전용 인터페이스

sysinfo\_demo는 Linux에서만 결과를 출력한다.

값을 예측한 뒤 REPORT.md에 기록합니다.

## 2부

# sysconf와 경계값

실행 시점에 시스템 경계값을 조회합니다.

# sysconf(3)의 역할

---

## 실행 시점 조회

컴파일할 때 정하지 않고  
실행할 때 값을 얻는다.

## 인자 형태

\_SC\_ 접두사 상수로  
조회할 값을 고른다.

## 반환형

long이다. 오류와 무한 값을  
-1로 함께 표현할 수 있다.

헤더: <unistd.h>. 인자는 int, 반환은 long입니다.

# sysconf의 반환값 계약

---

호출 직전에 `errno`를 0으로 만든다.

-1이고 `errno`가 0이면: 한계가 없거나 값을 정할 수 없다.

-1이고 `errno`가 0이 아니면: 조회 자체가 실패했다.

결론: -1 하나로 두 가지 뜻을 나눠 읽는다.

조회에 성공한 값을 얻은 뒤에 프로그램이 사용한다.

# 자주 쓰는 \_SC 상수

---

## **\_SC\_PAGESIZE**

메모리 페이지 크기.  
정렬과 mmap 계산에 쓴다.

## **\_SC\_NPROCESSORS\_ONLN**

지금 사용 가능한  
온라인 CPU 수다.

## **\_SC\_CLK\_TCK**

times(2) 값의 초당 틱  
수다. CPU 시간 환산에 쓴다.

# sysprobe: sysconf 읽기

```
static int probe(const char *label, int name)
{
    errno = 0;
    long value = sysconf(name);
    if (value == -1) {
        if (errno != 0) {
            perror(label);
            return 1;
        }
        printf("%s=unlimited\n", label);
        return 0;
    }
    printf("%s=%ld\n", label, value);
    return 0;
}
```

완성 예제: sysprobe.c

## 초기화

호출 직전에 errno를 0으로 만든다.

## 두 갈래

-1 뒤 errno의 값으로 실패와 무한을 나눈다.

## 재사용

probe 함수로 여러 값을 같은 계약으로 읽는다.

# 페이지 크기와 CLK\_TCK가 쓰이는 곳

---

페이지 크기: 메모리를 다루는 최소 단위다. 11주 mmap에서 다시 만난다.

CLK\_TCK: times(2)가 돌려준 틱을 초로 환산할 때 나눈다.

CPU 수: 작업 분배나 스레드 수를 정하는 근거가 된다.

값은 아키텍처·커널에 따라 다르다: x86\_64는 4096과 100이 흔하다.

같은 프로그램으로 여러 환경의 값을 비교하는 것도 과제다.

# pathconf와 fpathconf

---

## 파일시스템 경계값

같은 상수라도 대상  
파일시스템에 따라 다르다.

### **\_PC\_NAME\_MAX**

그 디렉토리에서 파일  
이름의 최대 길이이다.

### **\_PC\_PATH\_MAX**

경로 문자열의  
최대 길이이다.

헤더: <unistd.h>. 반환값 계약은 sysconf와 같습니다.

## 실습 2: 경계값 비교

```
./sysprobe
```

```
# 같은 값을 셸에서 확인 (Linux)
```

```
getconf PAGE_SIZE
```

```
getconf CLK_TCK
```

```
getconf NPROCESSORS_ONLN
```

```
getconf OPEN_MAX
```

```
# FD 한계는 3부에서 다룬다
```

```
ulimit -Sn
```

### getconf

sysconf가 주는 값을 셸에서 확인한다.

### ulimit -Sn

FD의 soft 한계를 확인한다.

### 두 번 확인

같은 값을 다른 문으로 맞춰 본다.

값을 예측한 뒤 REPORT.md에 기록합니다.

# 3부

## 자원 한계

getrlimit과 setrlimit로 프로세스의 상한을 다룹니다.

# getrlimit과 setrlimit

---

## **struct rlimit**

rlim\_cur는 soft 한계,  
rlim\_max는 hard 한계다.

## **getrlimit**

현재 값을 구조체로 얻는다.  
성공 0, 실패 -1이다.

## **setrlimit**

조건을 만족하면 값을 바꾼다.  
거부되면 -1과 EPERM이다.

헤더: <sys/resource.h>. 성공 0, 실패 -1입니다.

# soft 한계와 hard 한계

---

soft: 지금 강제되는 한계. 넘으면 호출이 실패하거나 시그널이 온다.

hard: soft가 올라갈 수 있는 상한이다.

일반 사용자는 soft를 hard까지 올릴 수 있다.

hard를 낮추면 일반 사용자는 되돌릴 수 없다.

RLIM\_INFINITY는 무제한을 나타내는 특별한 값이다.

# 주요 RLIMIT 종류

---

## **RLIMIT\_NOFILE**

이 프로세스가 열 수 있는  
FD 개수의 상한이다.

## **RLIMIT\_FSIZE / CPU**

파일 크기와 CPU 시간의  
상한이다. 초과하면 시그널.

## **RLIMIT\_AS / STACK / NPROC**

주소 공간, 스택, 사용자  
프로세스 수의 상한이다.

# limit\_show: getrlimit 출력

```
struct rlimit rl;
if (getrlimit(RLIMIT_NOFILE, &rl) == -1) {
    perror("getrlimit");
    return 1;
}
printf("%-7s", "nofile");
if (rl.rlim_cur == RLIM_INFINITY)
    printf(" soft=unlimited");
else
    printf(" soft=%ju", (uintmax_t)rl.rlim_cur);
/* hard도 같은 방식으로 출력한다 */
```

## 무제한 표시

RLIM\_INFINITY는  
unlimited로 표시한다.

## 자료형

rlim\_t를 uintmax\_t로  
변환해 %ju로 출력한다.

## 비교 표

여러 자원의 soft·hard를  
한 번에 살펴본다.

# 한계에 도달하면 무슨 일이 생기나

---

FD 개수 초과: `open`이 -1을 돌려주고 `errno`는 `EMFILE`이다.

파일 크기 초과: `write`가 `EFBIG`이고 `SIGXFSZ`가 갈 수 있다.

CPU 시간 초과: `SIGXCPU`가 먼저 오고 계속 쓰면 `SIGKILL`이다.

주소 공간 초과: `mmap`이나 메모리 할당이 `ENOMEM`으로 실패한다.

오류 반환과 시그널, 두 형태의 결과가 함께 존재한다.

# nofile\_demo: FD 한계 실험

```
int fd = open("/dev/null", O_RDONLY);
if (fd == -1) {
    if (errno == EMFILE)
        break;          /* 한계 도달 */
    perror("open");
    return 1;
}
fds[opened++] = fd;

/* 실험 뒤 정리 */
while (opened > 0)
    close(fds[--opened]);
```

완성 예제: nofile\_demo.c

## 실험 설계

빈 장치 파일을 열 수 있을 때까지 반복한다.

## EMFILE

FD 상한에 도달하면 이 오류로 끝난다.

## 자원 반환

실험 뒤 열린 FD를 모두 닫는다.

# setrlimit로 soft 한계 바꾸기

```
struct rlimit rl;
if (getrlimit(RLIMIT_NOFILE, &rl) == -1) {
    perror("getrlimit");
    return 1;
}
/* hard를 넘는 요청은 hard로 제한한다 */
rlim_t target = want > rl.rlim_max
               ? rl.rlim_max : want;
struct rlimit next = { target, rl.rlim_max };
if (setrlimit(RLIMIT_NOFILE, &next) == -1) {
    perror("setrlimit");
    return 1;
}
```

완성 예제: nofile\_demo --raise

## 상향 범위

일반 사용자는 hard를  
넘어 올릴 수 없다.

## 적용 범위

이 프로세스와 이후  
자식들에만 적용된다.

## 변경 확인

getrlimit으로 실제 값을  
다시 읽어 출력한다.

# 실습 3: FD 한계 관찰

```
./limit_show | head -1
./nofile_demo
./nofile_demo --raise 64
./nofile_demo --raise 4096

# 셸에서 soft만 바꿔 비교 (hard 이하)
bash -c 'ulimit -Sn 64 && ./nofile_demo'
bash -c 'ulimit -Sn 4096; ./nofile_demo'
```

값을 예측한 뒤 REPORT.md에 기록합니다.

## 측정

opened가 soft 값 근처에서 멈추는지 확인한다.

## 변경 실험

--raise 64와 4096으로 soft를 바꿔 비교한다.

## 셸 비교

ulimit -Sn도 같은 결과를 내는지 본다.

# 4부

## /proc 관찰 창

표준 API 바깥에서 Linux의 상태를 관찰합니다.

# /proc 파일시스템

---

커널과 프로세스 상태를 파일처럼 읽는 가상 파일시스템이다.

표준이 아니다: Linux 고유의 관찰 창이다.

프로그램의 판단 근거보다 진단과 스크립트에 쓴다.

읽을 때마다 커널이 그 순간의 상태를 만들어 준다.

지난 주의 `opendir·fgets` 조합으로 그대로 읽을 수 있다.

# /proc의 주요 경로

---

## **cpuinfo / meminfo**

CPU와 메모리의  
전체 요약 정보다.

## **loadavg**

1, 5, 15분 부하 평균과  
작업 수를 한 줄로 준다.

## **/proc/self/...**

이 프로세스의 상태,  
한계, 열린 FD를 가리킨다.

자세한 목록은 `proc(5)`에 있습니다.

# proc\_probe: /proc/self 관찰

```
FILE *f = fopen("/proc/self/status", "r");
if (f == NULL) {
    fprintf(stderr, "/proc/self/status: %s\n",
            strerror(errno));
    return 1;
}
char line[256];
while (fgets(line, sizeof line, f) != NULL)
    if (strncmp(line, "VmRSS:", 6) == 0 ||
        strncmp(line, "Threads:", 8) == 0)
        fputs(line, stdout);
fclose(f);
```

## 읽기 방법

3주차의 stdio 함수로  
일반 파일처럼 읽는다.

## 필터

필요한 줄만 골라  
화면에 남긴다.

## fd 세기

/proc/self/fd를 opendir로  
읽어 개수를 센다.

완성 예제: proc\_probe.c

# 표준 API와 /proc의 역할 분담

---

## 표준 API

uname·sysconf·getrlimit는  
이식성 있는 근거다.

## /proc

Linux의 진단·관찰 창이다.  
더 많은 정보를 담는다.

## 선택 기준

판단은 표준 API,  
진단 스크립트는 /proc로.

같은 값을 두 창에서 확인하는 것이 오늘의 관점입니다.

# 실습 4: /proc 관찰

```
./proc_probe  
  
# 같은 정보를 명령으로 확인 (Linux)  
grep -E 'VmRSS|Threads' /proc/self/status  
cat /proc/loadavg  
ls /proc/self/fd | wc -l  
grep 'open files' /proc/self/limits  
  
# 전체 요약은 선택  
head -3 /proc/meminfo
```

## 일치 확인

proc\_probe 결과와 명령 출력을 줄 단위로 맞춘다.

## fd 개수

명령마다 개수가 조금씩 다른 이유를 설명한다.

## limits

soft·hard가 getrlimit 결과와 같은지 본다.

값을 예측한 뒤 REPORT.md에 기록합니다.

# 확인 퀴즈

---

sysconf가 -1을 돌려줄 때 두 경우는 무엇이고 어떻게 구분하는가?

일반 사용자가 soft 한계와 hard 한계에서 바꿀 수 있는 범위는?

RLIMIT\_NOFILE에 도달한 open의 errno 값은?

sysconf(\_SC\_OPEN\_MAX)가 알려주는 값은 무엇과 같은가?

프로그램의 판단 근거로 /proc 대신 표준 API를 쓰는 이유는?

# 실습 정리와 제출

---

sysprobe 결과를 uname·getconf 출력과 항목별로 비교한다.

limit\_show의 soft·hard를 ulimit·/proc/self/limits와 비교한다.

nofile\_demo로 EMFILE까지의 개수와 한계 변경 전후를 기록한다.

proc\_probe 결과를 셸 명령 출력과 줄 단위로 대조한다.

C 소스, REPORT.md, 실행 로그를 제출한다.

선택 확장: sysprobe에 pathconf 항목 추가

# 참고 자료

---

Linux man-pages: `uname(2)`, `sysinfo(2)`, `getpagesize(2)`

Linux man-pages: `sysconf(3)`, `pathconf(3)`, `times(2)`

Linux man-pages: `getrlimit(2)`, `open(2)`, `signal(7)`

Linux man-pages: `proc(5)`

수업은 Ubuntu 일반 사용자 환경을 기준으로 합니다.